



计算机图形学 Project 2 实验报告

光照模型与光线追踪

课程	COMP130018.01 计算机图形学
姓名	刘卓鑫、朱劲舟
学号	24300240170
提交日期	2026 年 5 月 3 日

一、实验概述

本次 PJ 实现了一个基于光线投射（Ray Casting）与递归光线追踪（Ray Tracing）的渲染器。代码框架基于 C++ 提供，需要补全的核心部分包括：

1. **Phong 光照模型**：点光源照明计算、漫反射与镜面反射着色；
2. **几何体求交**：无穷平面（Plane）、三角形（Triangle）、几何变换（Transform）；
3. **递归光追**：反射光线与阴影光线；
4. **抗锯齿（扩展）**：抖动采样与高斯滤波下采样。

二、任务 1 —— Phong 光照模型

2.1 点光源（PointLight::getIllumination）

点光源与方向光的区别在于：点光源有位置，且光强随距离平方衰减。给定场景中的一点 $\mathbf{p}$ ，实现如下：

$$\mathbf{d} = \mathbf{pos} - \mathbf{p}, \quad d = \|\mathbf{d}\|, \quad \hat{L} = \frac{\mathbf{d}}{d}$$

光强衰减公式（ α 为 `_falloff`）：

$$I_{point} = \frac{I_{color}}{1 + \alpha \cdot d^2}$$

代码中直接计算向量长度作为 `distToLight`，然后将差向量归一化得到 `toLight`，方便后续阴影测试时比较命中距离与光源距离。

2.2 Phong 着色 (Material::shade)

Phong 模型将光照分为漫反射和镜面反射两部分（环境光在 `traceRay` 层面单独处理）：

漫反射 (Lambert 模型)：

$$I_{diffuse} = k_d \cdot \max(\hat{L} \cdot \hat{N}, 0) \cdot I_{light}$$

当 $\hat{L} \cdot \hat{N} \leq 0$ 时光源在切平面背面，漫反射和镜面反射均不计算。

镜面反射 (Phong 模型)：

$$\hat{R} = 2(\hat{L} \cdot \hat{N})\hat{N} - \hat{L}$$

$$I_{specular} = k_s \cdot \max(\hat{R} \cdot \hat{E}, 0)^s \cdot I_{light}$$

其中 $\hat{E} = -\hat{d}_{ray}$ 为从交点指向相机的方向， s 为光泽度 (shininess)。 s 越大高光越集中，表面越光滑。

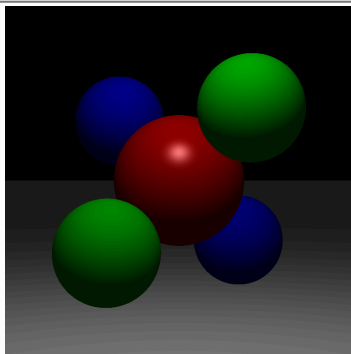
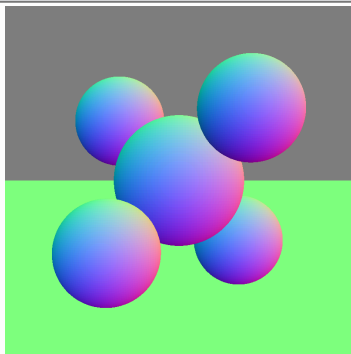
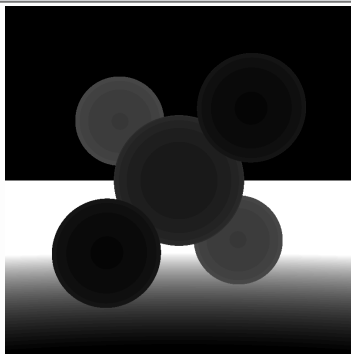
最终返回 $I_{diffuse} + I_{specular}$ （逐通道相乘光强）。

2.3 traceRay 中的 Phong 汇总

在 `Renderer::traceRay` 中，如果光线命中场景，则：

1. 计算环境光贡献： $k_d \odot L_{ambient}$ ；
2. 遍历所有光源，累加 `shade(...)` 的结果；
3. 如启用阴影，每个光源的贡献先经过阴影测试再累加。

场景 1 (球体 + 平面) 渲染结果：

彩色渲染	法线图	深度图
		

可以看到红色大球上有明显的高光（specular），蓝绿小球和白色平面上也能看到漫反射的光照变化，整体与参考图吻合。

三、任务 2 —— 光线投射

3.1 平面求交 (Plane::intersect)

平面方程为 $\mathbf{P} \cdot \hat{\mathbf{n}} = d$ ，光线为 $\mathbf{P}(t) = \mathbf{o} + t\mathbf{d}$ ，代入得：

$$t = \frac{d - \mathbf{o} \cdot \hat{\mathbf{n}}}{\mathbf{d} \cdot \hat{\mathbf{n}}}$$

当 $|\mathbf{d} \cdot \hat{\mathbf{n}}| < \epsilon$ （约 10^{-8} ）时光线平行于平面，无交点。命中后将 Hit 的法向量设为平面法线 $\hat{\mathbf{n}}$ 。

3.2 三角形求交 (Triangle::intersect)

采用矩阵法求解重心坐标，将问题转化为线性方程组：

$$\underbrace{\begin{bmatrix} \mathbf{a} - \mathbf{b} & \mathbf{a} - \mathbf{c} & \mathbf{d} \end{bmatrix}}_A \begin{bmatrix} \beta \\ \gamma \\ t \end{bmatrix} = \mathbf{a} - \mathbf{o}$$

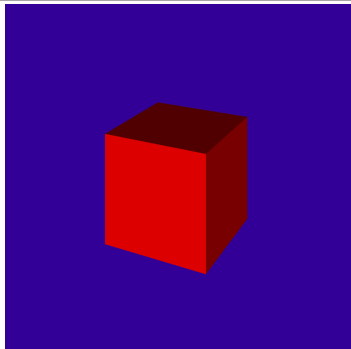
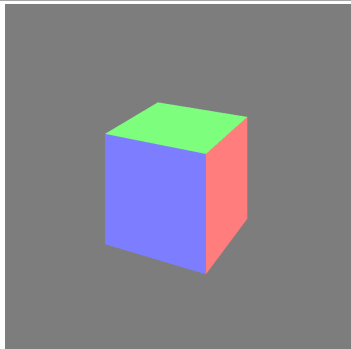
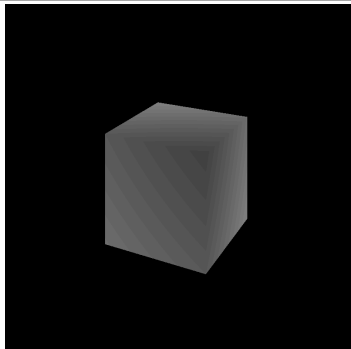
用 `Matrix3f::inverse()` 求 A^{-1} 解出 $[\beta, \gamma, t]$ ，再得 $\alpha = 1 - \beta - \gamma$ 。

命中判定条件： $-t > t_{min}$ 且 $t < h.t$ （更近的交点）； $-\alpha, \beta, \gamma \in [0, 1]$ （点在三角形内）。

命中后，法线通过重心坐标插值三顶点法线得到，实现平滑着色：

$$\hat{N} = \text{normalize}(\alpha \hat{n}_0 + \beta \hat{n}_1 + \gamma \hat{n}_2)$$

场景 2（三角网格立方体）渲染结果：

彩色渲染	法线图	深度图
		

立方体的三个可见面法线方向不同，在法线图中呈现明显的颜色区分。

3.3 几何变换 (Transform::intersect)

Transform 类将子对象从局部坐标变换到世界坐标，求交时反过来把光线变换到局部坐标空间（避免对整个网格顶点做变换）：

$$\mathbf{o}_{local} = M^{-1} \mathbf{o}_{world}, \quad \mathbf{d}_{local} = M^{-1} \mathbf{d}_{world}$$

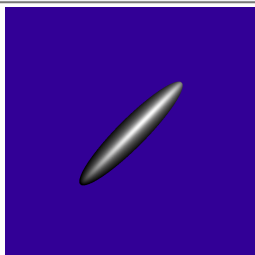
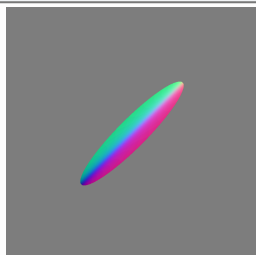
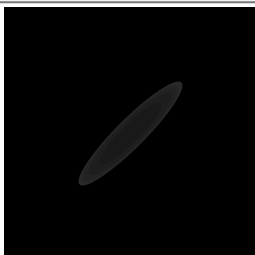
注意： $\mathbf{d}_{local}$ 不做归一化。由于参数化方式相同，局部空间中求出的 t 值直接等于世界空间中的 t ，不需要额外换算。

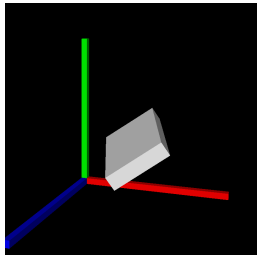
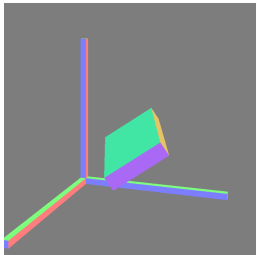
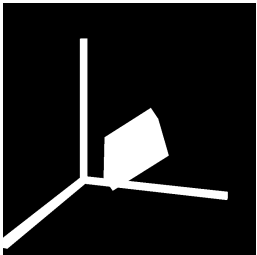
命中后，法线需要用逆转置矩阵变换回世界空间：

$$\hat{N}_{world} = \text{normalize} \left((M^{-1})^T \hat{N}_{local} \right)$$

构造时预先缓存 M 、 M^{-1} 、 $(M^{-1})^T$ 避免重复计算。

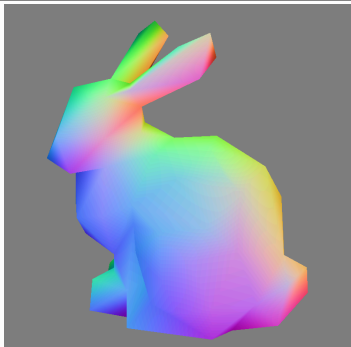
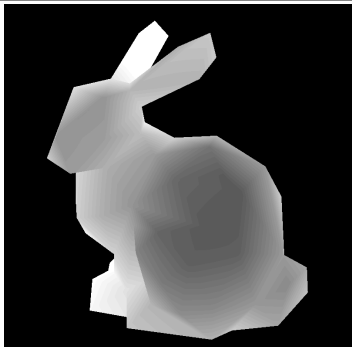
场景 3（变换后的椭球体）和场景 4（坐标轴）渲染结果：

场景	彩色渲染	法线图	深度图
Scene 3（椭球）			

场景	彩色渲染	法线图	深度图
Scene 4 (坐标轴)			

Scene 3 是对球体同时做 45° Z 轴旋转和 Y 轴方向 0.2 倍缩放得到的椭球，可以看到法线变换正确（椭球上的高光位置符合预期）。Scene 4 包含多个 Transform 嵌套的立方体构成坐标轴，三个轴分别为红绿蓝色。

场景 5 (Stanford Bunny, 200 面) 渲染结果：

彩色渲染	法线图	深度图
		

Bunny 由 200 个三角面片组成，两盏方向光照亮，法线图能看出 200 面时表面比较粗糙，多边形感较明显。

四、任务 3 —— 光线追踪与阴影投射

4.1 阴影光线

在对每个光源调用 `shade()` 之前，如果开启了 `-shadows` 参数，则从命中点 $\mathbf{p}$ 向光源方向发射一条阴影光线：

$$\text{shadowRay} = (\mathbf{p} + \epsilon \hat{L}, \hat{L})$$

如果该光线在到达光源之前 ($t < d_{light}$) 与场景中任意物体相交，则当前点处于该光源的阴影中，跳过该光源的贡献。

$\epsilon = 10^{-3}$ 的偏移是为了避免光线与自身所在的表面相交（自阴影 / shadow acne 问题）。

4.2 递归反射

当材质有镜面颜色 ($k_s > 0$) 且 `bounces > 0` 时，从命中点沿完美反射方向发射递归光线：

$$\hat{R} = \hat{d} - 2(\hat{d} \cdot \hat{N})\hat{N}$$

$$I_{total} = I_{direct} + k_s \odot \text{traceRay}(\text{Ray}(\mathbf{p} + \epsilon\hat{R}, \hat{R}), \epsilon, \text{bounces} - 1)$$

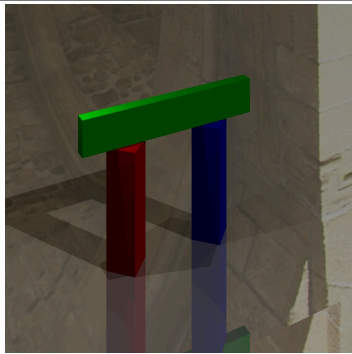
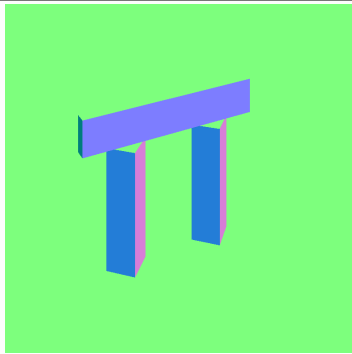
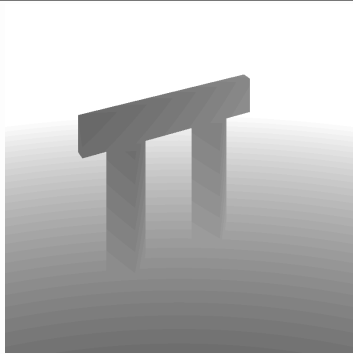
即使 `max_bounces = 0` 也会正常执行光线投射，只是不会产生反射。背景色通过 `SceneParser::getBackgroundColor(dir)` 获取，场景 6/7 配置了 CubeMap，未命中的光线会返回对应方向的天空盒颜色。

场景 6（1k Bunny，反射 + CubeMap 背景）：



镜面材质的兔子身上清晰反射出了背后教堂的 CubeMap，地面（Plane）同样具有一定镜面性，能看到兔子的倒影。递归深度为 4，兔子腹部与腿部之间的互相反射也被正确计算。

场景 7（Arch，阴影 + 反射 + CubeMap 背景）：

彩色渲染 (bounces=4, shadows)	法线图	深度图
		

场景中有一盏点光源（falloff=0.7，衰减较快）和一盏方向光。开启阴影后，红蓝两根柱子在地面上产生了清晰的投影，横梁下方也有阴影。地面同样有弱镜天性，可以看到柱子的淡淡倒影。

五、扩展 —— 抗锯齿

在较高分辨率下，几何边缘和镜面高光处会出现明显的锯齿。本次实现了两种互相配合的抗锯齿方法。

5.1 抖动采样 (-jitter)

对每个像素重复 16 次采样，每次对 NDC 坐标加入在 $[-0.5, 0.5]$ 像素宽度范围内均匀分布的随机偏移 $(\Delta x, \Delta y)$ ，最后取平均：

$$\bar{C} = \frac{1}{16} \sum_{i=1}^{16} \text{traceRay}(\text{generateRay}(x + \Delta x_i, y + \Delta y_i))$$

这种方法由 Pixar 提出，相比均匀超采样，随机抖动能更好地把锯齿转化为噪声，在视觉上更容易被人眼接受。

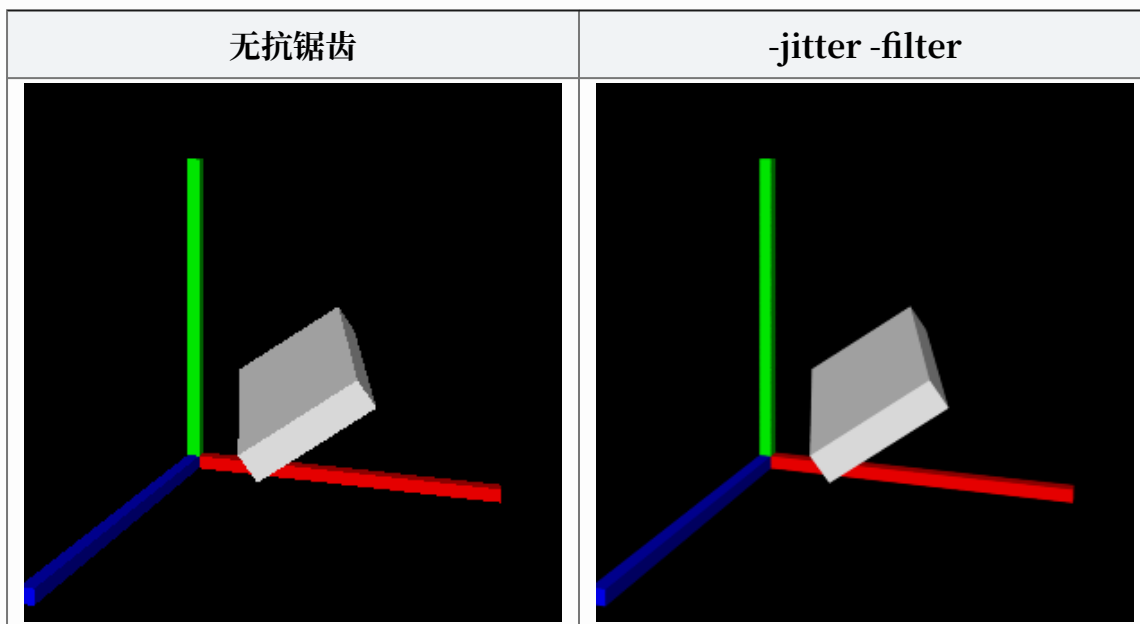
5.2 高斯滤波 (-filter)

以 3 倍分辨率 ($3w \times 3h$) 渲染整张图像，再用 3×3 高斯核下采样回原分辨率 ($w \times h$)：

$$G = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

高斯滤波是低通滤波器，能抑制高频的锯齿噪声，同时相比简单平均保留了更多的边缘细节（高斯权重对中心像素赋予更高权重）。

抗锯齿效果对比（场景 4， 300×300 ）：



可以看到，无抗锯齿时坐标轴的斜边（尤其是蓝色轴和红色轴的端部）有明显的阶梯状锯齿；开启抗锯齿后边缘平滑了很多，整体视觉质量有明显提升。

六、实验总结

本次 PJ 让我对光线追踪的完整流程有了比较深入的理解，以下几点是实现过程中印象比较深刻的地方：

1. **Transform 求交的 t 值一致性**：一开始没注意到局部光线方向不能归一化，导致 Transform 场景的深度图出错，后来仔细推导才明白参数化的一致性问题。
2. **自相交 offset 的选取**：阴影和反射光线都需要对起点做微小偏移， ϵ 太大会丢失近距离阴影，太小会产生 shadow acne。最终选用 10^{-3} ，在所有测试场景中效果都比较稳定。

3. **法线插值 vs 平面法线：**三角形的法线插值（用重心坐标混合三顶点法线）比直接用面法线效果好很多，在 Bunny 这类低面数模型上尤为明显，法线图的渐变也更平滑。
4. **高斯滤波的效率：**3 倍上采样会使渲染时间增加到约 9 倍，加上 16 次 jitter 采样总计约 144 倍的计算量。在 800×800 的场景 6 上测试时间较长，实际提交时对要求分辨率的 AA 效果图建议在较小分辨率下演示。